



LEARN

LOOPS IN LINUX (BASH)

Loops help automate repetitive tasks by executing a block of code multiple times.

1. For Loop

Use When: You know the number of times you want to loop (fixed iterations).

Syntax:

```
for variable in list
do
    commands
done
```

Example:

```
for fruit in apple banana mango
do
    echo "I like $fruit"
done
```

Output:

```
I like apple
I like banana
I like mango
```



For Loop with Range

```
for i in {1..5}
do
    echo "Number: $i"
done
```

Also:

```
for (( i=1; i<=5; i++ ))
do
    echo "Number: $i"
done
```

2. While Loop

Use When: You don't know how many times you'll loop in advance. It runs while the condition is true.

Syntax:

```
while [ condition ]
do
    commands
done
```

Example:

```
count=1
while [ $count -le 5 ]
do
    echo "Count: $count"
```



```
((count++))  
done
```

3. Until Loop

Use When: You want the loop to run until the condition becomes true.

Syntax:

```
until [ condition ]  
do  
    commands  
done
```

Example:

```
x=1  
until [ $x -gt 5 ]  
do  
    echo "x: $x"  
    ((x++))  
done
```

4. Loop Control Statements

Command	Purpose	Example
break	Exits the loop immediately	if [\$i -eq 3]; then break; fi
continue	Skips current iteration and continues	if [\$i -eq 3]; then continue; fi



FUNCTIONS IN LINUX (BASH)

Functions allow you to group commands and reuse them. This avoids code repetition and enhances modularity.

1. Creating a Function

Syntax 1:

```
function_name () {  
    commands  
}
```

Syntax 2:

```
function function_name {  
    commands  
}
```

2. Calling a Function

Just write the function name:

```
function_name
```

3. Function with Parameters

Use \$1, \$2, ... to access parameters.

Example:

```
greet() {  
    echo "Hello, $1"  
}
```



```
greet "Kanchan"
```

Output:

```
Hello, Kanchan
```

4. Returning Values from a Function

- Use **return** for exit status (0–255)
- Use **echo** to send data back

Exit Status Example:

```
check_even() {  
  if (( $1 % 2 == 0 )); then  
    return 0  
  else  
    return 1  
  fi  
}
```

```
check_even 4  
if [ $? -eq 0 ]; then  
  echo "Even"  
else  
  echo "Odd"  
fi
```

Echo Return Example:

```
sum() {  
  result=$(( $1 + $2 ))
```



```
echo $result  
}  
  
total=$(sum 5 7)  
echo "Total is $total"
```

5. Using local Variables in Functions

```
myfunc() {  
    local name="Alice"  
    echo "Name is $name"  
}
```

PRACTICE

1. Print even numbers from 1 to 10 using a for loop:

```
for i in {1..10}  
do  
    if (( i % 2 == 0 )); then  
        echo "$i is even"  
    fi  
done
```

2. Write a function to multiply two numbers:

```
multiply() {  
    product=$(( $1 * $2 ))  
    echo "Product is: $product"
```



```
}
```

```
multiply 4 6
```

3. Print the contents of a directory using a while loop:

```
index=0
files=(*)
while [ $index -lt ${#files[@]} ]
do
    echo "File: ${files[$index]}"
    ((index++))
done
```

4. Prompt user until they type “exit”:

```
input=""
while [ "$input" != "exit" ]
do
    read -p "Type 'exit' to stop: " input
done
```

5. Function to check if a number is positive, negative, or zero:

```
check_number() {
    if [ $1 -gt 0 ]; then
        echo "Positive"
    elif [ $1 -lt 0 ]; then
```



```
    echo "Negative"
else
    echo "Zero"
fi
}

check_number -3
```

FAQ

Q1. Can I define functions anywhere in the script?

Yes, but it's best to define all your functions at the top of the script and call them later.

Q2. What's the difference between return and echo in functions?

- **return**: gives exit status (success or failure, 0–255)
- **echo**: prints output you can capture and use

Q3. How do I capture a function's output?

Use command substitution:

```
result=$(function_name args)
```

Q4. How can I loop over files in a directory?

```
for file in /path/to/folder/*
do
    echo "Processing $file"
```




```
done
```

Q5. How do I loop through lines in a file?

```
while read line
do
    echo "Line: $line"
done < filename.txt
```

Q6. Can I nest functions or loops?

Yes. You can have functions inside functions, or loops inside loops.

```
outer() {
    for i in 1 2
    do
        echo "Outer: $i"
        for j in a b
        do
            echo " Inner: $j"
        done
    done
}
```

Q7. How to debug loops or functions?

- Use **set -x** to enable debugging
- Add **echo** to print variable values